Cours : Le principe Ouvert/Fermé (Open/Closed Principle – OCP)

1. Rappel du contexte SOLID

Les principes **SOLID** guident la conception orientée objet pour produire un code maintenable, extensible et robuste. Le **OCP** est le deuxième principe : il complète le SRP en favorisant l'évolution du code sans modifier le comportement existant.

2. Définition du OCP

"Les entités logicielles (classes, modules, fonctions) doivent être **ouvertes à l'extension**, mais **fermées à la modification**." — Bertrand Meyer
Cela signifie qu'un composant doit pouvoir évoluer sans être modifié : on ajoute du comportement sans toucher au code existant.

3. Lien entre SRP et OCP

Le **SRP** impose qu'une classe ait une seule responsabilité. Le **OCP** complète ce principe : si cette responsabilité doit évoluer, on étend la classe plutôt que de modifier son comportement.

Si une classe fait bien une seule chose (SRP), mais qu'on veut qu'elle la fasse différemment selon le contexte, on crée une nouvelle classe dérivée ou une nouvelle implémentation plutôt que de changer la classe d'origine.

4. Héritage vs Interface dans le cadre du OCP

Deux approches respectent le OCP : l'héritage et l'implémentation d'interfaces. Elles permettent d'étendre sans modifier, mais avec des implications différentes.

Héritage :
• Extension simple du comportement.
• Réutilisation du code parent.
• Couplage fort entre classe mère et classe fille.
• Limité à une seule hiérarchie.

Interface ou composition :
• Approche plus souple, sans dépendance forte.
• Favorise le polymorphisme et l'injection de dépendances.
• Permet d'étendre les comportements sans affecter le code existant.

Conclusion : le OCP ne dicte pas le moyen (héritage ou interface), mais l'objectif : *étendre sans modifier*.

5. Exemple conceptuel

Un système de calcul de remises :
■ Mauvaise pratique : une seule classe avec des conditions pour chaque type de remise.
■ Bonne pratique : une interface **DiscountPolicy** implémentée par plusieurs classes (**LoyaltyDiscount**, **PromoDiscount**, etc.). On ajoute une nouvelle remise sans modifier le code existant.

6. Conclusion

Étendre plutôt que modifier.
Le OCP encourage un code stable, évolutif et modulaire. Combiné au SRP, il favorise une architecture souple où les évolutions se

traduisent par l’ajout de nouveaux composants plutôt que la modification de ceux déjà testés.